

Mahdi Heydari

Protocol Engineer & Security Researcher

abramsymons@gmail.com Muscat, Oman · Remote (GMT+4) abramsymons.github.io github.com/abramsymons
x.com/0xmahdi_

Protocol engineer and security researcher. I design decentralized systems where ordering is separated from execution: replicas agree on a log, and each app replays it to reach the same state. I've applied that idea since 2018: BrightID's node consensus, the Muon oracle network, the Zellular sequencer, and the Zex exchange. I believe simplicity is the root of both security and performance, so I start with the simplest prototype that proves a protocol, then make it production-ready. Security researcher with a \$30,000 Aptos bounty for a network-halting consensus bug.

SECURITY RESEARCH

Blockchain bug bounties · Independent

- **Aptos:** \$30,000 bounty for a High-severity consensus vulnerability that could halt the network. Three more valid findings were closed as duplicates.
- **Sui:** reported 8 vulnerabilities. 6 denial-of-service issues were confirmed as duplicates, 4 of them reachable by unauthenticated network peers. 2 more are under review.
- **Zex custody (internal audit):** contributed to the pre-launch security audit of the custody stack: threshold key generation and signing, and multi-chain deposit and withdrawal handling.
- **Muon:** reported 14 security and design flaws in the node and its threshold-signature library, including a rogue-key attack on distributed key generation and a single-node denial of service of key generation.

EXPERIENCE

Zex

2024 – present

Founder & Protocol Architect (2024 – 2025) → **Tech Lead & Reviewer** (2025 – present)

Multi-chain exchange that runs as a replicated state machine on a decentralized sequencer.

- Built Zex as the first app on the Zellular stack, combining sequencing, decentralized custody (ZexPorta) and an orderbook engine (Cora).
- **Simple protocol first:** wrote the first matching-engine prototype in 2024. App-specific transactions, with only buy, sell and cancel on the hot path, let it reach about 100K orders/s in Python.
- **Performance back to target:** returned in late 2025, after a period of feature work had pushed throughput down to around 10K tx/s, and brought it back above 100K by removing bottlenecks in both architecture and code across the engine and the API services: O(1) cancels (15× faster), backpressure between stages, an export pipeline about 3× faster with 75% less overhead, Kafka prefetch and event-loop fixes (17% → 2% throughput loss, 280 → 32 ms gaps between batch groups), Rust serialization, a batch API, key caching and chunked sequencer sends. Added an end-to-end throughput benchmark.
- **Then a lower-level language (2026):** with the protocol proven, a few small changes made an exact C port practical. Verified against the Python engine batch by batch, it reaches up to 6M orders/s on simple loads and about 1M on a realistic deep order book, around 25× faster.
- **Production consensus (2026):** started `vseq`, which runs Firedancer's Alpenglow consensus as a standalone ordering service with client-verified BLS finality proofs. Designed it and directed an AI-assisted implementation in C.
- **Multi-chain threshold custody (ZexPorta):** designed how one FROST validator group controls funds on Bitcoin (per-user Taproot keys), EVM and Tron (CREATE2 deposits, on-chain Schnorr verification) and Solana and Aptos (native Ed25519), with an independent key co-signing every withdrawal. Explored shielded Zcash custody. Pre-mainnet.
- Built transaction authentication in the core engine: signing-key verification, replay protection, and a fork mechanism for deterministic protocol upgrades.
- **Wallet-free onboarding:** designed OpenID Connect login verified by every replica, with no custodial provider, the same approach Aptos and Sui use for keyless accounts, and adapted passkeys to serverless apps.
- Reviewed 90+ pull requests from 6 engineers across the engine, custody and transaction format.

Zellular · Founder & Protocol Architect

2023 – 2025

Decentralized infrastructure for running dApps as replicated state machines. Merged into Zex in 2025.

- Designed an architecture in which dApps rely on decentralized services for consensus and custody, so developers write only app logic and never manage keys or chain integrations.
- **Sequencing:** designed the Zellular BFT protocol (leader-based, no proposer rotation or gossip, BLS-aggregated finality proofs, leader dispute and switchover). Built the prototype for 100K tx/s through batching, with sub-second finality, and ran it as an EigenLayer AVS with about 10 testnet operators. Core developer of `zsequencer` (250+ commits) and sole author of the `zellular.py` app SDK.
- **Threshold custody, from protocol to product:** wrote the [pyfrost protocol specification](#) (DKG with verifiable secret sharing, rogue-key defense, and an analysis showing how naive complaint handling lets two colluders exclude honest nodes, which led to an identifiable-cheating DKG) and designed the library's node, key-generator and aggregator roles. Then designed `zbtc` (an EigenLayer-secured Bitcoin bridge) and `EigenSync` (verifying EigenLayer operator signatures on any EVM chain) as prototypes, and generalized them into a multi-chain custody service for exchanges, restaking platforms and payment apps, now ZexPorta at Zex. For production, replaced `pyfrost`'s own cryptography with the Zcash Foundation's audited FROST, keeping the Python networking layer.
- **EigenLayer tooling:** lead author of [eigensdk-python](#) (the Python port of EigenLayer's Go SDK, including slashing and rewards) and `avs-subgraph`. Co-built `eigensdk-js` and the Python reference AVS.

Muon Network

2021 – 2023

Architect (2021 – 2022) → Technical Lead (2022 – 2023)

Decentralized oracle network using threshold signatures. Grew to 3,400+ testnet operators and a 1,500+ node mainnet. [muon.net](#)

- Conceived and designed Muon's architecture as a decentralized price oracle for DEUS/Symmio, later generalized for any dApp.
- Chose threshold signatures from day one, when MPC was still rare in web3: hundreds of nodes produce one Schnorr signature that costs the same to verify on-chain however many nodes sign, where multisig gas grows with every signer.
- **Latency:** moving signing from the JavaScript threshold implementation to FROST cut signature latency from about 7 seconds to under a second.
- **Modular security:** designed three layers dApps can combine. Threshold signatures give cryptographic security. An optimistic layer gives economic security: warrantor nodes lock collateral behind each response, supervisor nodes dispute bad data, and a DAO slashes dishonest warrantors. A shield lets an independent party re-verify each event and co-sign; if it misbehaves it can only stop the flow, never forge data.
- While reviewing the security of the large JavaScript node, decided a lean Python rewrite was safer than auditing it. Led about 6 engineers building it (`muon-frost-py`), teaching them the FROST protocol, along with the optimistic node and the explorer. Reviewed 77 core-node PRs.
- Started the developer documentation, and built a price-feed app and the node backup and deployment tooling.

BrightID · Core Team Developer

2018 – 2023

Social identity network that proves each user has one account. 73K+ verified users, integrated by 15 apps including Gitcoin Grants. [brightid.org](#)

- Primary architect and developer of the [BrightID node](#), which stores the social graph and serves verifications to apps. Wrote or reviewed most of its code (600+ commits).
- Designed the node consensus layer. Sequencing every user connection on Ethereum mainnet would have cost too much gas, so before sidechains and L2s existed we launched [IDChain](#), a dedicated proof-of-authority chain (geth Clique). Nodes replay graph operations in block order, so every node converges on the same social graph.
- Led anti-sybil research: SybilRank variants, a group-based ranking algorithm and a simulation platform for modeling attacks on the graph. Implemented the Bitu verification model (about 12K users verified by 2022).
- Designed and built blind-signature verifications, so apps can confirm a user is verified without linking the user's accounts across apps. Chose the scheme and libraries and implemented it in the node and the client.
- Co-designed and built the IDChain ecosystem: explorer, DEX, bridge to Ethereum and a gas faucet for verified users. Implemented social recovery and the verification contracts.

Freelance Software Engineer · Freelancer.com

2013 – 2018

- Co-ran a two-person development team: 220+ completed projects for international clients, averaging 4.8 / 5 across 228 reviews.
- Python automation, data processing and web scraping, web services and APIs, and server administration. Moved into Ethereum smart contracts and token sales from 2017.

Software Developer · City transit systems, Shiraz

2010 – 2013

- Built the back end, front end and driver settlement for the city bus fare system, used by hundreds of thousands of citizens paying with contactless cards on every bus.
- Made driver payments fail-safe with two independent calculation paths: one from passenger tap records, one computed by the bus reader at each driver shift change. Payment was held until both matched, so every discrepancy was found and fixed before money moved.
- Built an automatic vehicle location (AVL) system that tracks buses in real time, produces fleet reports, and announces next-bus arrival times at stations and on board.

AWARDS

1st place, EigenLayer AVS MicroHacks

2024 · \$12K prize, 100+ builders. Python and JavaScript EigenLayer SDKs.

100,000 EIGEN ecosystem grant

2024 · Eigen Foundation, for Zellular's EigenLayer work.

EDUCATION

B.S. Software Engineering

Shiraz University, 2010

SKILLS

Protocols

BFT consensus Sequencing EigenLayer AVS Oracles

Cryptography

FROST DKG Schnorr BLS OIDC / JWT

Security

Consensus review Network DoS Threshold-crypto attacks

Workflow

AI-assisted development AI-assisted auditing

Languages

Python TypeScript JavaScript Solidity

Chains

Ethereum Bitcoin Solana Aptos Sui

Infrastructure

PostgreSQL Redis Kafka Docker